

Assignment 6: EduStream — Course Learning Platform

Pending



Objective

Build a **FastAPI-based Course Learning Platform** where teachers can create and manage courses, while students can enroll and access course content only after successful enrollment.

The assignment should demonstrate:

- Role-Based Access Control (RBAC)
- Class-Based Dependencies
- Nested Pydantic Schemas
- Path Parameters
- User-Specific Access Control
- Enrollment Validation

Technical Requirements

- **Framework:** FastAPI
- Roles:
 - teacher
 - student
- Use `get_current_user` dependency
- Access control must be implemented using **dependency classes**
- Course modules must use a **nested schema**
- Enrollment must be unique for each user and course

Data Models

User

- id
- email
- role

Course

- id
- title
- description
- price
- instructor_id

Module

- title
- content
- order_index

Enrollment

- user_id
- course_id
- enrolled_at

API Routes

POST /courses

Teacher Only

Create a new course with multiple modules.

Required fields:

- Title
- Description
- Price
- Modules

Modules must be submitted using a **nested Pydantic schema**.

Example:

```
{
  "title": "FastAPI Fundamentals",
  "description": "Learn FastAPI from basics",
  "price": 1999,
  "modules": [
    {
      "title": "Introduction",
      "content": "Introduction to FastAPI",
      "order_index": 1
    },
    {
      "title": "Routing",
      "content": "Working with FastAPI routes",
      "order_index": 2
    }
  ]
}
```

Validation:

- Only teachers can create courses.
- `price >= 0`

GET /courses

Return the public list of available courses.

Support optional filtering using:

`instructor_name`

Example:

/courses?instructor_name=Rahul

This route should be publicly accessible.

POST /courses/{id}/enroll

Enroll the currently authenticated student in a course.

Requirements:

- Use the course ID as a **path parameter**.
 - Use `get_current_user`.
 - Duplicate enrollment for the same user and course must be blocked.
 - Course must exist before enrollment.
-

GET /courses/{id}/content

Protected route to access course modules and content.

Requirements:

- User must be authenticated.
 - Student must be enrolled in the course.
 - Students without active enrollment must not be allowed to access the course content.
 - Access control should be handled using a **class-based dependency**.
-

PATCH /courses/{id}

Teacher Only

Update course details.

Allow updates to:

- Description
- Price

Validation:

- Only teachers can update courses.
 - `price >= 0`
 - Return `404 Not Found` if the course does not exist.
-

Nested Pydantic Schema

Modules must be included inside the Course schema.

Example:

```
class ModuleCreate(BaseModel):
    title: str
    content: str
    order_index: int

class CourseCreate(BaseModel):
    title: str
    description: str
    price: float = Field(ge=0)
    modules: list[ModuleCreate]
```

Role-Based Access Control

Implement RBAC for:

Teacher

Can:

- Create courses
- Update courses

Student

Can:

- View courses
- Enroll in courses
- Access content after enrollment

- Access content after enrollment

Access restrictions should be enforced through dependencies.

Validation & Business Rules

- Only teachers can create courses.
 - Only teachers can update courses.
 - Course price must be greater than or equal to 0.
 - Students cannot access course content without enrollment.
 - Duplicate enrollment is not allowed.
 - Course ID must be handled using path parameters.
 - Enrollment must belong to the authenticated user.
-

Error Handling

Handle appropriate errors for:

- Course not found
- Unauthorized course creation
- Unauthorized course update
- Duplicate enrollment
- Student not enrolled
- Invalid price
- Invalid request data

Use appropriate HTTP status codes such as 400, 401, 403, 404, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Teacher course creation
 - Nested module creation
 - Public course listing
 - Instructor filtering
 - Student enrollment
 - Duplicate enrollment
 - Course content access
 - Unauthorized content access
 - Teacher course update
 - RBAC and dependency classes
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- requirements.txt
- README.md
- All files required to run the project

ZIP File Name

StudentName_EduStream.zip

Example:

RahulKumar_EduStream.zip

Important

- Submit only **one ZIP file.**

- Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- Role-Based Access Control
- Class-based dependency implementation
- Nested Pydantic schemas
- Enrollment logic
- Duplicate enrollment handling
- Protected course content access
- Path parameter usage
- Validation and error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI learning platform using RBAC, class-based dependencies, nested schemas, path parameters, enrollment validation, and protected content access.**

Upload Assignment